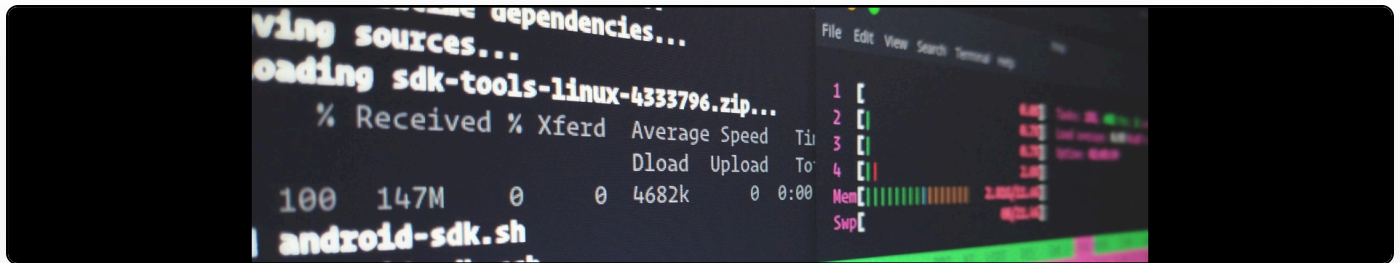


HTC Wildfire Stage 1 Soft-Root

Cite as: Martin Paul Eve, "HTC Wildfire Stage 1 Soft-Root", *eve.gd: Martin Paul Eve*, August 15, 2010, <https://doi.org/10.59348/vvye0-qq070>.



 An HTC Wildfire displaying an Android Superuser permission request, beside a handwritten note reading Martin Eve

UPDATE 2013-06-30: I'm afraid that I've had to remove the below files as my host thinks they are a virus. Great. Anyway, this method has easily been surpassed by now.

This is a break from my customary blog posts on Thomas Pynchon and my university research to present a sample of one my other research interests in the realm of computer science and information security. Google has, for a fair while now, been distributing their stripped down version of the Linux operating system -- Android -- on mobile devices. These devices are capable of running as fully fledged Linux distributions but for the fact that manufacturers lock down the phones and make it incredibly difficult to gain administrative privileges on the devices. As such, I have begun investigating ways by which to circumvent this ridiculous restriction of users' rights on their own devices; as the recent US Supreme Court ruling sensibly decreed: the devices are owned by the end-users, the end-users should be able to control what is run on such systems and circumventing the protection mechanisms on a device one owns is neither illegal, nor protected by the DMCA. Recently, a group dubbed "The Android Exploit Crew" released an extremely clever piece of code for the Android operating system which exploits the hotplug system. Essentially, it manages to install itself as a callback function upon enable/disable of any hotplug device (wifi/bluetooth) which is executed with escalated privileges. The original exploit copies itself to a new binary in /system/bin, the flash-memory filesystem which has been remounted read-write, and

which is owned by the root account and has the setuid bit set. Now: the recent HTC device, the Wildfire (codenamed: Buzz), has an interesting system of protection on the flash memory -- NAND protection. This means that, despite the read-write remount of the /system filesystem, any write to this area will result in the system spiralling out of memory, refusing the write and then rebooting. Obviously, this means that the exploit, in its original form, results in a crash and reboot. I have now modified this exploit to perform differently so that it will work on the Wildfire. I anticipate that the best usage for my work is as follows, which I may attempt to implement if I have time: Setup application (can we run at startup?) checks for existence of /system/bin/su

If not existent, it unpacks su binary and exploid binary to /sqlite_stmt_journals

Runs exploit

Rebind mount /dev/block/mtdblock3 to /sqlite_stmt_journals/binmount

Symlinks all binaries from /sqlite_stmt_journals/binmount/bin to /sqlite_stmt_journals/newbin

Copies su to /sqlite_stmt_journals/newbin

Chmods/chowns /sqlite_stmt_journals/newbin to a safe combo

Mounts /sqlite_stmt_journals/newbin over the top of /system/bin

su will now function correctly Anyway, I have achieved all this manually and now have at /system/bin/ the su binary and, linked into this, the Superuser.apk application! Obligatory screenshot of barnacle wifi tether requesting superuser permissions attached. So, anyway, to reproduce this, grab these files (source at the end of the post if you want to recompile):

<http://www.martineve.com/wildfirestage1root/su>

<http://www.martineve.com/wildfirestage1root/busybox>

<http://www.martineve.com/wildfirestage1root/Superuser.apk>

<http://www.martineve.com/wildfirestage1root/exploid> ANY STEPS YOU TAKE FROM HEREON ARE YOUR OWN UNDERTAKING. I ACCEPT NO RESPONSIBILITY FOR A BRICKED DEVICE, EVEN THOUGH I PERSONALLY HAD NO PROBLEMS. Setup adb

Push all the files to /sqlite_stmt_journals/

Execute: adb shell

cd /sqlite_stmt_journals

./exploid Toggle your wifi on and off Back at shell, execute: mkdir binmount

mkdir newbin

chmod 755 ./busybox

./exploid

```
./busybox mount -r -t yaffs2 /dev/block/mtdblock3 ./binmount
```

```
./busybox ln -s /sqlite_stmt_journals/binmount/bin/* /sqlite_stmt_journals/newbin/
```

```
./busybox cp ./su ./newbin/
```

```
./busybox mount --bind /sqlite_stmt_journals/newbin /system/bin
```

```
./busybox cp ./Superuser.apk /data/app/
```

```
./busybox rm ./exploid
```

```
./busybox rm ./su You now have a rooted HTC Wildfire... until you reboot. Source files:
```

<http://www.martineve.com/wildfirestage1root/exploid.c>

<http://www.martineve.com/wildfirestage1root/makefile>

<http://forum.xda-developers.com/showthread.php?t=682828>